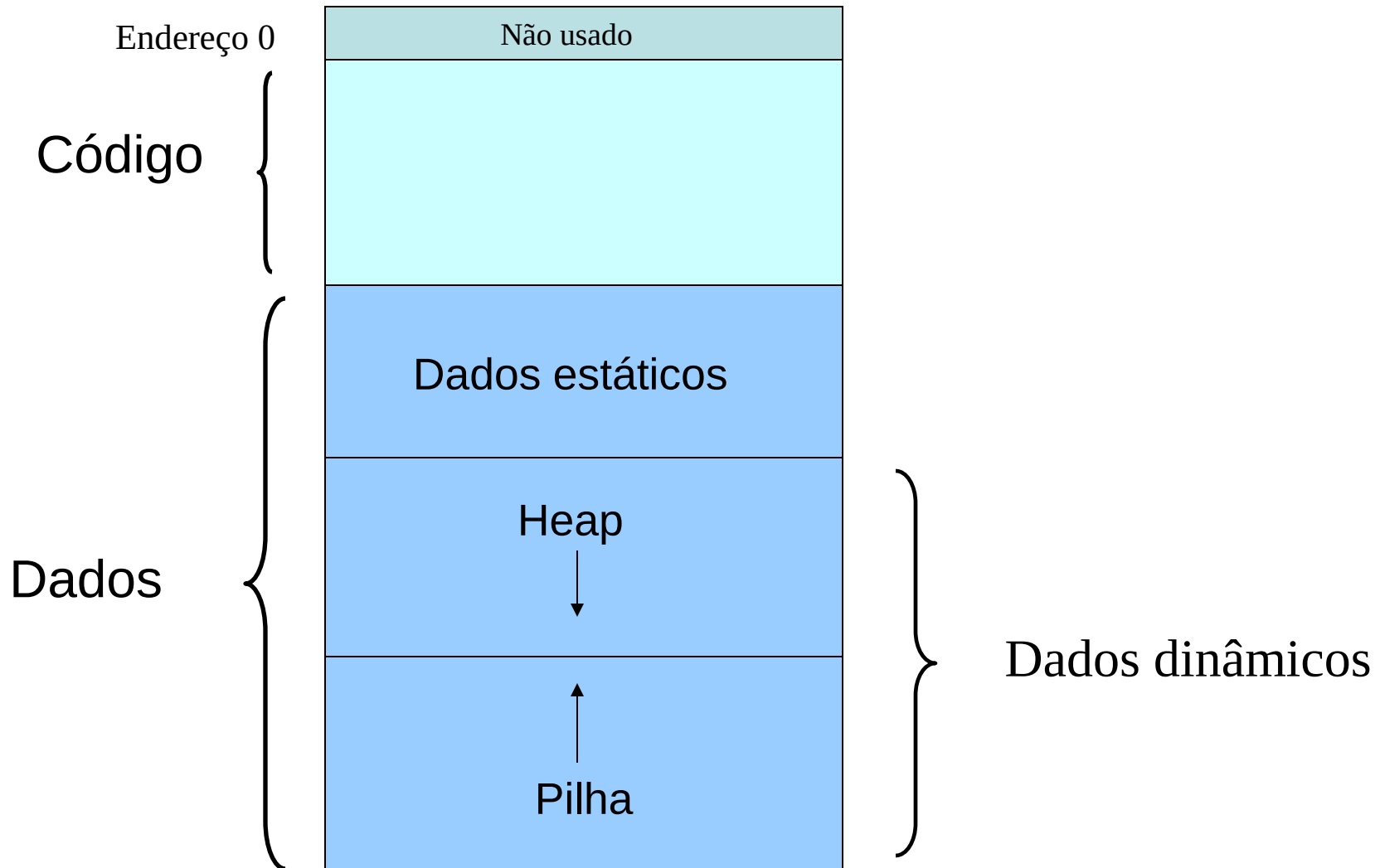

Ambiente em Tempo de Execução

(Runtime environment)

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

Organização da Memória



Dados Estáticos

A área de memória é reservada no início da execução do programa e liberada apenas no fim de sua execução (e.g. variáveis globais e variáveis locais declaradas com o modificador *static* em linguagem C).

Dados Dinâmicos

Pilha – Área de armazenamento temporário onde é armazenado o registro de ativação das funções.

Heap – Área reservada para alocação dinâmica, permite ao programador alocar e liberar espaços de memória quando necessário (e.g. áreas de memória reservadas pelas funções *malloc* e liberadas pela função *free* em linguagem C).

Registro de Ativação

As informações necessárias para execução de uma função/procedimento são gerenciadas utilizando um bloco de memória chamado registro de ativação, fazem parte do registro de ativação: parâmetros, endereço de retorno e variáveis locais.

Chamadas de Funções/Procedimentos

```
int fat (int n)
{
    if (n <= 1)
        return 1;
    return n * fat(n-1);
}
```

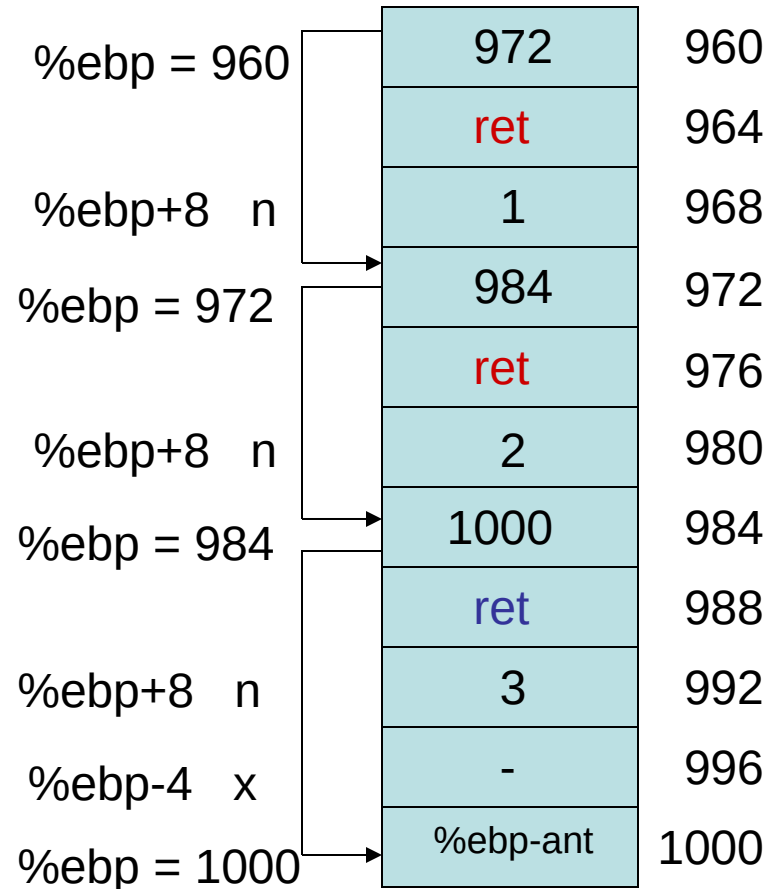
```
fat:
    pushl %ebp
    movl %esp, %ebp
    movl 8(%ebp), %ebx
    cmpl $1, %ebx
    jl L1:
    movl $1, %eax
    movl %ebp, %esp
    popl %ebp
    ret
L1: subl $1, %ebx
    pushl %ebx
    call fat
    subl $4, %esp
    movl 8(%ebp), %ebx
    imull %ebx, %eax
    movl %ebp, %esp
    popl %ebp
    ret
```

```
fat:
    pushl %ebp
    movl %esp, %ebp
    movl 8(%ebp), %ebx
    cmpl $1, %ebx
    jl L1:
    movl $1, %eax
    jmp L3
L1: subl $1, %ebx
    pushl %ebx
    call fat
    movl 8(%ebp), %ebx
    imull %ebx, %eax
L3: movl %ebp, %esp
    popl %ebp
    ret
```

Chamadas de Funções/Procedimentos

```
int fat (int n)
{
    if (n <= 1)
        return 1;
    return n * fat(n-1);
}

Int main()
{
    int x;
    x = fat(3);
}
```



Garbage Collector

(Coletor de Lixo)

Coletores de lixo são uma forma automática de gerenciamento de memória, responsável por liberar a memória que foi alocada no *heap* e não é mais referenciada. Foi proposto por John McCarthy em 1959 para simplificar o gerenciamento de memória da linguagem LISP. Evita problemas como:

- Dangling Pointers;
- Erros de dupla liberação;
- Vazamento de memória (*space leaks*).

Reference Counting

(Contadores de Referência)

- Reserva alguns bits junto ao dado para indicar quantas referências existem ao objeto.
- Simples, mas inclui um overhead em cada operação de atribuição.
- Problemas para lidar com referências circulares.

Mark and Sweep

- Suspende a execução do programa e utiliza os apontadores para determinar quais os objetos estão em uso.
- Objetos que não são referenciados são considerados lixo e sua área de memória é liberada.